

Klausur-Pilotenschein Programmierung 1

PIB-PR1 im Wintersemester 2025/26

Professor Martin Burger

2. Februar 2026

Name: _____ Vorname: _____

Matrikelnummer: _____ Studiengang: ☐ PIB ☐ DFIW

Semester, in dem die Prüfungsvorleistung erbracht wurde (z. B. WiSe 25/26): _____

Anzahl der als Hilfsmittel verwendeten DIN A4 Blätter (maximal 1): _____

Aufgabe	1	2	3	4	5	Summe
Max. Punktzahl	6	8	6	4	10	34
Davon erreicht						

Die maximal erreichbare Punktzahl entspricht 100 % und damit der Note 1,0. Es gibt nur ganze Punkte, keine Halb- oder Zwischenwerte. Die Aufgaben sind so gestaltet, dass Sie die 100 % in der verfügbaren Zeit erreichen können, wenn Sie die in der Lernveranstaltung vermittelten Konzepte rasch anwenden und strukturiert vorgehen.

Hinweise für einen fairen und reibungslosen Ablauf (bitte zuerst lesen). Beginnen Sie anschließend mit den Aufgaben. Unterschreiben Sie bitte auf der nächsten Seite.

- **Teilnahmevoraussetzungen:** Eine Teilnahme ist nur möglich, wenn Sie in SIM angemeldet sind und alle erforderlichen Prüfungsvorleistungen erbracht haben. Andernfalls ist eine Teilnahme leider nicht möglich. Eine dennoch erfolgende Bearbeitung wird als Täuschungsversuch gewertet.
- **Gesundheit:** Mit Beginn der Bearbeitung bestätigen Sie, dass Sie sich gesundheitlich in der Lage fühlen, die Prüfung abzulegen.
- **Schreibmaterial:** Bitte verwenden Sie ausschließlich dokumentenechte Stifte. Bleistifte, Radierstifte, Frixion-Stifte und ähnliche Schreibgeräte sind nicht zulässig.
- **Deckblatt / Identifikation:** Bitte füllen Sie das Deckblatt vollständig und gut lesbar aus. Prüfen Sie gegebenenfalls auch die Schreibweise des voreingetragenen Namens. Bei Blankoexemplaren: Tragen Sie Ihre Matrikelnummer ab Seite 3 in die Kopfzeile jeder ungeraden Seite ein, bevor Sie mit den Aufgaben beginnen.

- **Lesbarkeit / Korrekturen:** Bitte schreiben Sie klar und deutlich. Teile, die nicht bewertet werden sollen, streichen Sie bitte deutlich erkennbar durch.
- **Antwortbereiche (Teil 2):** Bitte tragen Sie die Lösungen zu Teil 2 ausschließlich in die vorgesehenen Antwortbereiche ein. Einträge außerhalb dieser Bereiche können leider nicht bewertet werden. Dies gilt auch für einzelne Teilaufgaben.
- **Zusatzblätter:** Sollten Sie weitere Blätter benötigen, melden Sie sich bitte bei den Aufsichtspersonen. Sie erhalten dann offizielle Ergänzungsblätter.
- **Heftung / Verweise:** Bitte lassen Sie die Klausur geheftet. Wenn Sie Lösungen auf Zusatzblättern schreiben, vermerken Sie dies bitte deutlich bei der entsprechenden Aufgabe.
- **Zugelassene Hilfsmittel:** Es sind ausschließlich die im Prüfungsplan angekündigten, handschriftlich beschriebenen DIN-A4-Blätter zulässig. Bitte streichen Sie zu Beginn alle unbeschrifteten Bereiche deutlich durch und notieren Sie Ihren Namen sowie Ihre Matrikelnummer auf jedem Blatt.
- **Abgabe der Hilfsmittel:** Die Hilfsmittel sind Bestandteil der Klausur und werden mit abgegeben. Ein Mitnehmen aus dem Raum ist nicht gestattet.
- **Kurzzeitiges Verlassen des Raums:** Wenn Sie den Raum vorübergehend verlassen möchten, melden Sie sich bitte. Es darf sich immer nur eine:r von Ihnen außerhalb des Raums aufhalten. Geben Sie Ihre Klausur dazu bei den Aufsichtspersonen ab. Der Zeitpunkt und die Dauer werden notiert.
- **Abgabe / Prüfungsende:** Eine vorzeitige Abgabe ist in den letzten 15 Minuten nicht mehr möglich. Bitte bleiben Sie nach Ablauf der Prüfungszeit ruhig sitzen, bis alle Klausuren eingesammelt wurden.
- **Elektronische Geräte:** Bitte schalten Sie Mobiltelefone und ähnliche Geräte aus und verstauen Sie diese in Ihrer Tasche oder Ihrem Rucksack (nicht in Hosentaschen oder Jackentaschen). Am Körper getragene Geräte sowie Geräusche, die von solchen Geräten ausgehen, werden als Täuschungsversuch gewertet. Ausgenommen sind zuvor angezeigte medizinische Geräte.
- **Kommunikation:** Bitte führen Sie keine Gespräche untereinander. Richten Sie Fragen bitte ausschließlich an die Aufsichtspersonen. Gespräche bzw. inhaltliche Kommunikation mit anderen Personen als den Aufsichtspersonen gelten als Täuschungsversuch.
- **Konsequenzen bei Täuschungsversuchen:** Ein Täuschungsversuch hat den sofortigen Ausschluss von der Klausur zur Folge und wird dem Prüfungsamt gemeldet.

Vielen Dank für Ihre Mithilfe. Viel Erfolg!

Mit meiner Unterschrift bestätige ich, dass ich die oben stehenden Hinweise zur Kenntnis genommen habe und mich zu ihrer Einhaltung verpflichte.

Unterschrift: _____

Eine nicht unterschriebene Klausur ist ungültig. Sie wird mit 0 Punkten bewertet.

Teil 1: Grundlagen

1. Aufgabe: Code-Tracing: Reise durch Java

Gesamt: (6)

Ihre Aufgabe: Bei den folgenden Code-Tracing-Aufgaben verfolgen Sie den Java-Code, um das Programmverhalten zu untersuchen. Jede Aufgabe besteht aus zwei Multiple-Choice-Fragen: Die erste bezieht sich auf das *beobachtbare Verhalten* des Programms, die zweite auf die Erklärung des *Programmablaufs*.

- i **Beobachtbares Verhalten:** Kreuzen Sie jeweils die *eine* Antwort an, die der tatsächlichen Ausgabe auf der Konsole bzw. der Reaktion des Programms bei der Ausführung am besten entspricht!
- ii **Beschreibung des Ablaufs:** Kreuzen Sie die *eine* Antwort an, die den Ablauf des Programms am besten beschreibt!

Sie erhalten die volle Punktzahl für eine Code-Tracing-Aufgabe nur, wenn Sie *sowohl* die Frage zum Verhalten (i) *als auch* die zum Ablauf (ii) richtig beantworten. Ist nur eine der beiden Antworten korrekt, erhalten Sie keine Punkte. Das Gleiche gilt, wenn Sie keine oder mehrere Antworten pro Frage ankreuzen.

Tipp: Verlieren Sie sich nicht in einzelnen Code-Tracing-Aufgaben. Wenn Sie nach drei Minuten noch keine Lösung gefunden haben, markieren Sie die Aufgabe als “übersprungen” und gehen Sie zur nächsten über. Falls noch Zeit bleibt, können Sie am Ende zu den übersprungenen Aufgaben zurückkehren.

(a) Verstehen Sie Ihre Variablen (Primitive & Referenzen)

(3)

Strings verhalten sich in Java anders als primitive Datentypen. Betrachten Sie diesen Code:

```
1 public class StringTestDrive {  
2     public static void main(String[] args) {  
3         String text = "hallo";  
4         text.toUpperCase();  
5         System.out.print(text);  
6     }  
7 }
```

i. **Beobachtbares Verhalten:** Welche Ausgabe erscheint auf der Konsole?

- ☐ null
- ☐ hallo
- ☐ HALLO
- ☐ Eine "Speicheradresse" in der Form String@1a2b3c.

ii. **Beschreibung des Ablaufs:** Welche Erklärung beschreibt den Sachverhalt am besten?

- ☐ String-Objekte sind unveränderlich (engl. immutable). Die Methode `toUpperCase()` verändert nicht das Originalobjekt, sondern erstellt einen neuen String und gibt diesen zurück. Der Wert von `text` bleibt unverändert, da der Rückgabewert nicht gespeichert wird.
- ☐ Die Methode `toUpperCase()` funktioniert nur, wenn der String vorher mit `new String("hallo")` explizit erstellt wurde. Bei String-Literalen wird die Optimierung des Compilers aktiv und verhindert Änderungen.
- ☐ Der Code enthält einen logischen Fehler: Die Methode müsste `void` sein und den String "in-place" (an Ort und Stelle) ändern, aber Java hat hier einen bekannten Bug in der Standardbibliothek.
- ☐ Die Ausgabe erfolgt gepuffert. Da `System.out.print` vor der internen Verarbeitung des Strings fertig ist, wird noch der alte Wert angezeigt. Mit `Thread.sleep()` würde HALLO erscheinen.

(b) Wie sich Objekte verhalten (Zustand & Verhalten)**(3)**

Betrachten Sie den folgenden Code, der eine Methode aufruft, die als Parameter eine Ganzzahl übergeben bekommt und basierend darauf eine Berechnung durchführt:

```
1 public class NumberMagicTestDrive {
2     public static void main(String[] args) {
3         int number = 5;
4         updateNumber(number);
5         System.out.println(number);
6     }
7
8     public static void updateNumber(int n) {
9         n = n * 2;
10    }
11 }
```

i. **Beobachtbares Verhalten:** Welche Ausgabe wird auf der Konsole ausgegeben, wenn dieser Code ausgeführt wird?

- ☐ 0
- ☐ 5
- ☐ 10
- ☐ In Zeile 8 liegt ein Compiler-Fehler vor, da die Variable `number` in der Methode `public static void updateNumber(int n)` nicht sichtbar ist.

ii. **Beschreibung des Ablaufs:** Welche Erklärung beschreibt den Ablauf des Programms am besten?

- ☐ Die Methode `updateNumber` ist **static**, daher kann sie nicht auf die lokale Variable `number` der `main`-Methode zugreifen. Java schützt Variablen automatisch vor Änderungen.
- ☐ Da die Methode **void** ist, wird das Ergebnis der Berechnung `n * 2` vom Garbage Collector sofort gelöscht, noch bevor es zugewiesen werden kann.
- ☐ Java übergibt primitive Datentypen als Wertkopie (Pass-by-Value). Die Methode `updateNumber` ändert nur ihre eigene, lokale Kopie `n`; die Variable im Aufrufer bleibt unberührt.
- ☐ Es handelt sich um einen Referenzfehler: Man müsste `updateNumber(&number)` aufrufen, damit die Speicheradresse übergeben wird und die Änderung wirksam wird.

Teil 2: Anwendungsaufgaben

2. Aufgabe: Clean Code, Refactoring & Softwaredesign

Gesamt: (8)

Diese Aufgabe prüft, ob Sie ein Verständnis für lesbaren Code haben und grundlegende Entscheidungen im Softwaredesign treffen können.

(a) **Clean Code: Refactoring**

(4)

Auch in kleinen Programmen ist lesbarer Code essenziell. Im folgenden Beispiel sehen Sie eine Methode, die zwar einwandfrei funktioniert, jedoch kryptische Namen und “magische Zahlen” verwendet:

```
1 public class PriceCalculator {
2     public double c(double p) {
3         return p * 1.19;
4     }
5 }
```

Ihre Aufgabe: Führen Sie ein Refactoring der oben gezeigten Klasse durch, um den Code lesbarer zu machen!

Wählen Sie dazu für jede Lücke den passenden *Lösungsbaustein* aus der folgenden Liste (A–F) aus und tragen Sie den entsprechenden Buchstaben in die Tabelle auf der nächsten Seite ein.

```
1 public class PriceCalculator {
2
3     _____ (1) _____
4
5     _____ (2) _____ {
6         return price * VAT_RATE;
7     }
8
9 }
```

Lösungsbausteine:

A `double mwst = 1.19;`

B `public double calc(double p)`

C `public double calculateGrossPrice(double price)`

D `return p * 1.19;`

E `public static final double VAT_RATE = 1.19;`

F `public void setPrice(double price)`

Wichtig: Jeder Baustein darf höchstens einmal verwendet werden.

Tragen Sie hier die entsprechenden *Buchstaben* aus der Liste der Lösungsbausteine von der vorherigen Seite ein:

Lücke	Ihre Lösung	Anmerkungen (optional)
(1)		
(2)		

(b) **Softwaredesign: Entwurfsentscheidung**

(4)

Szenario: Sie schreiben eine Klasse namens `MathHelper`, die rein mathematische Hilfsmethoden wie `add(int a, int b)` enthält. Diese Methoden führen Berechnungen ausschließlich anhand ihrer Parameter durch und benötigen keinen Zugriff auf Instanzvariablen.

Ihre Aufgabe: Entscheiden Sie, ob diese Methoden als `static` definiert werden sollten! Begründen Sie Ihre Wahl in einem knapp formulierten, aussagekräftigen Satz. Ihre Formulierung muss Ihre Entscheidung für oder gegen `static` nachvollziehbar darlegen.

3. Aufgabe: **Code-Repair & Debugging: Fakultät**

Gesamt: (6)

Mit dieser Aufgabe zeigen Sie, dass Sie fremden Code lesen, verstehen und reparieren können.

Eine Kommiliton:in wollte eine Methode schreiben, die das Produkt aller Zahlen von 1 bis n berechnet (für $n = 4$ also $1 \cdot 2 \cdot 3 \cdot 4 = 24$).

Leider wurde sie unterbrochen, sodass der Code **zwei logische Fehler** enthält. Das Programm liefert immer 0 oder läuft endlos:

```
1 class MathUtils {
2     public static int calculateProduct(int n) {
3         int product = 0;
4         int i = 1;
5
6         while (i <= n) {
7             product = product * i;
8             i -= 1;
9         }
10
11         return product;
12     }
13 }
```

Ihre Aufgaben:(a) **Fehlererkennung**

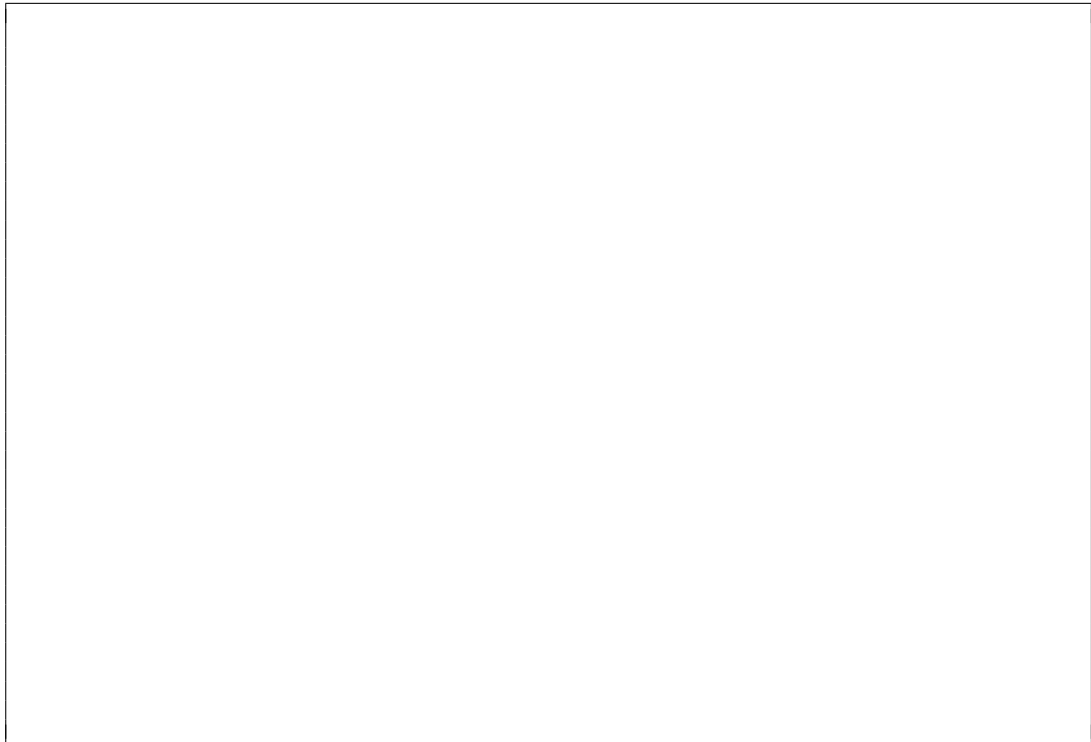
(2)

Kreisen Sie die *Zeilennummern* der zwei fehlerhaften Zeilen im obigen Code ein!

(b) Fehlerbehebung in der 1. fehlerhaften Zeile

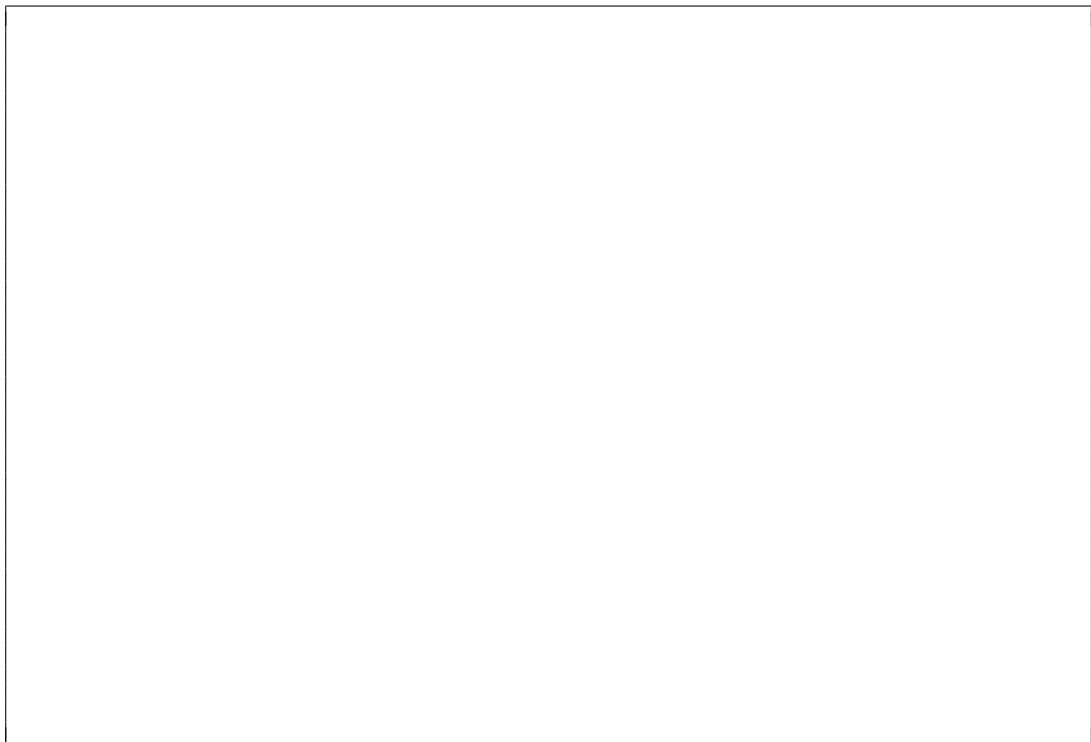
(2)

Tragen Sie Ihre korrigierte Version der *ersten* fehlerhaften Zeile in das folgende Feld ein!

**(c) Fehlerbehebung in der 2. fehlerhaften Zeile**

(2)

Tragen Sie Ihre korrigierte Version der *zweiten* fehlerhaften Zeile in das folgende Feld ein!



4. Aufgabe: Arrays & Schleifen: Summe berechnen

Gesamt: (4)

Ihre Aufgabe: Vervollständigen Sie die untenstehende Methode `calculateSum`! Die Methode soll die Summe aller **int**-Werte im übergebenen Array `numbers` berechnen und das Ergebnis zurückgeben:

```
1 public int calculateSum(int[] numbers) {  
2     int sum = 0;  
3  
4     for (int i = 0; i < _____(1)_____; i++) {  
5         sum = _____(2)_____;  
6     }  
7  
8     return sum;  
9 }
```

Wählen Sie dazu für jede Lücke den passenden Code-Baustein aus der Liste (A–F) aus und tragen Sie den entsprechenden Buchstaben in die Tabelle auf der nächsten Seite ein:

Code-Bausteine:

- A `numbers.size()`
- B `sum + numbers[i]`
- C `numbers.length()`
- D `sum + i`
- E `numbers.length`
- F `sum + numbers`

Wichtig: Jeder Baustein darf höchstens einmal verwendet werden.

Tragen Sie hier die entsprechenden *Buchstaben* aus der Liste der Code-Bausteine von der vorherigen Seite ein:

Lücke	Ihre Lösung	Anmerkungen (optional)
--------------	--------------------	-------------------------------

(1)

(2)

5. Aufgabe: **Programmlogik sortieren & anwenden**

Gesamt: (10)

In dieser Aufgabe beschäftigen wir uns mit der Ausgabe von Klausurergebnissen.

(a) **Programmlogik sortieren**

(6)

Unten sehen Sie den Code für die Methode `printStatus`, die basierend auf dem erreichten Prozentwert (`percentageValue`) eine entsprechende Nachricht ausgibt. Der Programmcode ist durcheinandergeraten.

```
// Schnipsel A
} else {
    System.out.println("Leider nicht bestanden.");
}
```

```
// Schnipsel B
public void printStatus(int percentageValue) {
```

```
// Schnipsel C
    if (percentageValue >= 40) {
```

```
// Schnipsel D
}
```

```
// Schnipsel E
    System.out.println("Bestanden!");
```

Ihre Aufgabe: Bringen Sie den Code in die richtige Reihenfolge, damit die Methode korrekt ausgeführt werden kann!

Notieren Sie dazu die Buchstaben der Schnipsel in der richtigen Reihenfolge (z. B. C - A - B ...):

(b) Programmlogik anwenden**(4)**

Nun wollen wir die in Teil (a) definierte Methode `printStatus` nutzen, um die Ergebnisse mehrerer Studierender auszugeben.

Ihre Aufgabe: Implementieren Sie die Logik der Methode, sodass für jedes Ergebnis im Array `results` die Methode `printStatus` aufgerufen wird! Fügen Sie dazu die noch fehlenden Anweisungen direkt in den folgenden Java-Code ein:

```
1  int[] results = { 44, 78, 40, 33, 91 };
2
3
4  for ( _____ ) {
5
6
7      _____;
8
9  }
```
